AetherWeave 2.0

Implementation Plan v2.0

1. วัตถุประสงค์ของแผนพัฒนา

เอกสารฉบับนี้อธิบายขั้นตอนการพัฒนา AetherWeave 2.0
ตั้งแต่ต้นแบบ (Prototype) ไปจนถึงระบบที่พร้อมใช้งานในระดับ Production-Like Environment

เป้าหมายหลัก:

- พัฒนา AI-Driven Adaptive Network Overlay
- สร้าง Context-Aware Routing Mechanism
- บังคับใช้ Zero Trust + Privacy-by-Design
- รองรับ Edge + Cloud Hybrid Deployment
- สร้างระบบที่สามารถทดสอบและสาธิตได้จริง
-

2. แนวทางการพัฒนา (Development Strategy)

AetherWeave 2.0 จะพัฒนาแบบ Phase-Based Iterative Development

แนวทางหลัก:

- Modular Architecture
- Microservice-based Deployment
- API-First Design
- Secure-by-Default

- Test-Driven Development (TDD)
- DevOps + CI/CD Integration

3. Phase 1 – Core SDN Infrastructure Prototype

เป้าหมาย สร้างโครงสร้างเครือข่ายพื้นฐานแบบ Software-Defined Network (SDN)

งานที่ต้องทำ

- ติดตั้ง SDN Controller (เช่น OpenDaylight / Ryu)
- สร้าง Virtual Network Topology (Mininet)
- พัฒนา Telemetry Collector
- สร้าง Basic Routing Policy Engine
- สร้าง Monitoring Dashboard

Deliverables

- SDN-Based Virtual Network Prototype
- Real-time Network Monitoring Dashboard
- Basic Policy Routing Implementation

4. Phase 2 – AI Traffic Intelligence Module

เป้าหมาย ทำให้เครือข่ายสามารถวิเคราะห์และทำนาย traffic ล่วงหน้าได้

งานที่ต้องทำ

- เก็บ Traffic Dataset
- พัฒนา Time-Series Forecasting Model (LSTM / Prophet)
- พัฒนา Congestion Prediction Module
- เชื่อม AI กับ SDN Controller
- สร้าง Dynamic Path Switching Logic

ตัวอย่างโครงสร้างโมดูล

```python
class AIPolicyEngine:

    def collect_traffic_data(self):
        pass

    def predict_congestion(self, history):
        pass

    def generate_policy(self, prediction):
        pass

    def update_sdn_controller(self, policy):
        pass
```

Deliverables

- AI Traffic Prediction Model
- Dynamic Routing Module
- Performance Benchmark Report

5. Phase 3 – Context-Aware Layer

เป้าหมาย พัฒนา Context-Based Network Policy System

งานที่ต้องทำ

- Define ContextProfile Schema
- สร้าง Context Collection API
- Implement Context Normalization
- พัฒนา Context-Based Policy Mapping
- Integrate Context ⟶ AI Policy Engine

ContextProfile Interface

```python
interface ContextProfile {
   userId: string
   deviceType: string
   applicationType: string
   activityLevel: number
   networkCondition: string
   timeOfDay: string
   timestamp: number
}
```

Deliverables

- Context API v1.0
- Context-Policy Mapping Engine
- Demo Scenario (e.g., Smart Campus Simulation)

6. Phase 4 – Security & Zero Trust Module

เป้าหมาย ทำให้ระบบปลอดภัยในระดับ Architecture

งานที่ต้องทำ

- Implement Zero Trust Authentication
- Integrate Role-Based + Attribute-Based Access Control
- Enforce TLS 1.3 Encryption
- Implement Micro-Segmentation
- Build Policy Enforcement Gateway

Deliverables

- Zero Trust Access Module
- Security Audit Report
- Penetration Testing Simulation

7. Phase 5 – Adaptive Control Plane Integration

เป้าหมาย รวม AI + Context + SDN ให้ทำงานร่วมกันแบบ Real-Time

งานที่ต้องทำ

- Implement Policy Orchestrator
- Create Intent-Based Networking API
- Develop Automatic Policy Update Trigger
- Implement Failover & Self-Healing Logic

Deliverables

- Integrated Adaptive Network Engine
- Real-Time Policy Adjustment Demo
- Self-Healing Simulation Test

8. Phase 6 – Edge Deployment & Scalability Testing

เป้าหมาย ทดสอบระบบในสภาพแวดล้อม Edge + Cloud Hybrid

งานที่ต้องทำ

- Deploy Edge Node Simulation
- Implement Federated Model Update (optional)

- Test Horizontal Scaling
- Perform Load Testing
- Evaluate Latency Improvement

Deliverables

- Edge Deployment Report
- Scalability Test Results
- Latency & Throughput Analysis

9. Testing Strategy

AetherWeave 2.0 จะใช้การทดสอบหลายระดับ:

- Unit Testing (AI, Policy Engine, Context API)
- Integration Testing (AI ↔ SDN ↔ Context)
- Load Testing (Traffic Spike Simulation)
- Fault Injection Testing (Controller Failure Simulation)
- Security Testing (Zero Trust + Encryption Validation)
- Performance Benchmarking

10. Deployment Plan

Environment

- Development
- Staging
- Production-Like Lab Environment

Deployment Model

- Container-based Deployment (Docker)
- Orchestrated via Kubernetes
- CI/CD Pipeline (GitHub Actions / GitLab CI)
- Infrastructure as Code (Terraform – optional)

11. Risk Management

ความเสี่ยง	แนวทางลดความเสี่ยง
AI Prediction Inaccuracy	Retraining + Fallback Static Routing
SDN Controller Failure	Redundant Controller Setup
Context Data Privacy Risk	Data Anonymization + Encryption
High Resource Usage	Edge Offloading + Resource Limiting
Integration Complexity	Modular Testing Strategy

12. Timeline Overview (High-Level)

Phase 1: 1–2 เดือน

Phase 2: 2 เดือน

Phase 3: 1–2 เดือน

Phase 4: 1 เดือน

Phase 5: 2 เดือน

Phase 6: 1–2 เดือน

รวมระยะเวลาประมาณ: 8–11 เดือน

(Prototype Demo สามารถทำได้ภายใน 4–5 เดือนแรก)

13. Success Metrics (KPIs)

- Traffic Congestion Reduction ≥ 25%
- Routing Efficiency Improvement ≥ 20%
- AI Prediction Accuracy ≥ 85%
- Self-Healing Recovery Time ≤ 2 seconds
- Zero Unauthorized Access Events
- Latency Reduction ≥ 15% (Edge Scenario)

14. สรุป

Implementation Plan นี้กำหนดขั้นตอนพัฒนา AetherWeave 2.0

จาก SDN Prototype ⟶ AI-Driven Adaptive Network ⟶ Context-Aware Secure Infrastructure

AetherWeave 2.0 ไม่ได้แทนที่ Internet เดิม

แต่เป็น Intelligent Overlay Layer ที่เพิ่มความสามารถด้าน:

- การปรับตัว (Adaptation)
- การทำนาย (Prediction)
- ความปลอดภัย (Security)
- ความเข้าใจบริบท (Context Awareness)

ระบบนี้สามารถพัฒนาในระดับมหาวิทยาลัย

และขยายสู่ระดับงานวิจัยหรืออุตสาหกรรมได้ในอนาคต